UNIVERSITEIT VAN PRETORIA
UNIVERSITY OF PRETORIA
YUNIBESITHI YA PRETORIA

Department of Computer Science

COS 132 - Imperative Programming
Study Unit 4: Program Modularity - Functions

Contents

4.1 Introduction

In this study unit program modularity is considered. Modular programs bring code together that provides and particular subtask or function. Where the functions representing the subtasks is placed in the code enhances the modularity of the code.

Depending on where and how the functions representing the subtasks are defined will determine the level to which the separation of concerns takes place, either in the file where the main function is defined or in separate header and implementation files.

An appendix has been included on compiling and linking. Both compiling and linking of a single file is discussed as well as how you would craft a makefile to aid in the compiling and linking process.

Before the discussion of functions, we discuss the special type `void` that is specifically used as return types in functions. A second discussion that is important to have, is the choosing of function names to enhance code readability and understandability.

4.2 The `void` type

The void type is a unique type in C++. Unlike other types that represent data such as integers, floating points, or characters, void represents the absence of type. This characteristic of void allows it to serve multiple purposes within the language, each catering to different aspects of programming in C++.

You can't declare a variable of type void. When used in the declaration of a pointer (more on this later), void specifies that the pointer is "universal".

4.3 Introduction to Functions

C++ programs can be divided into a series of identifiable subtasks, known as functions. Functions are used to group and name statements that:

- belong together, or
- are to be repeated.

A function in C++ can be either a void function or a function that returns a value. Consider the following program that asks the user for two numbers and displays their sum:

```
#include <iostream>
using namespace std;

int main() {
    int num1, num2 answer = 1;

    cout << "Enter first number: ";
    cin >> num1;
```

```
    cout << "Enter second number: ";
    cin >> num2;

    answer = num1 + num2;

    cout << answer << endl;
    return 0;
}
```

Listing 4.1: A Simple Program Without Additional Functions

Now, every time you want to determine the sum of two user entered numbers, you have to repeat this code. This leads to a lot of duplicated code and code that is difficult to follow. Would it not be easier to write the code once and call it by name each time we need it? This would make the program much more readable.

In C++ we use functions to achieve this. You have already seen the `main` function.

4.4 Naming Conventions

Every function in your program will perform one or more actions. It is important that the name of the function is descriptive of the actions that the function performs. This makes your program more readable as it helps developers quickly understand what a function does, what type of data it deals with, and how it is intended to be used within the broader context of the program.

Function names traditionally start with a verb, since the function performs some action. We usually adopt the Camel Case convention. Camel case starts with a lowercase letter, and subsequent words are joined together with the first letter of each new word capitalised.

Compare the following functions:

```
double process() {
    double num1;
    cout << "Enter radius: ";
    cin >> num1;

    double result = 3.14159 * r * r
    return result;
}
```

Listing 4.2: Poorly named function

```
double calculateCircleArea() {
    double num1;
    cout << "Enter radius: ";
    cin >> num1;

    double result = 3.14159 * r * r
    return result;
```

```
}
```

Listing 4.3: Well-Named function

The function name `process` in Listing: 4.2 is vague and provides no indication of what the function is processing. Without looking at the implementation, it's impossible to guess the function's purpose, whether it's processing data, performing calculations, or something entirely different.

`calculateCircleArea` in Listing: 4.3, clearly communicates what the function does: it calculates the area of a circle. Someone reading this code would immediately understand what the function is doing without having to read the code.

Following naming conventions and having well-named functions communicate the purpose of the code and makes your program more readable.

4.5 Variations in function syntax

All functions are uniquely identified by a function name. Refer to Section 4.4 to ensure you name your functions in an intelligible manner. Furthermore, functions will define a return type, or the absence of a return type and a parameter list or an absence of a list of parameters. There are therefore four variations of functions that can be defined in C++. These are:

- functions with no return type and no parameter list.
- functions with no return and and a parameter list
- functions with a return type and no parameter list
- functions with a return type and a parameter list

The sections that follow will provide an overview of how functions are defined when no return type or parameter list is given (Section 4.5.1), when function does accept a parameter list(Section 4.5.3), and when a function has a return type (Section 4.5.4).

4.5.1 Void, parameter-less functions

When used as a function return type, the `void` keyword specifies that the function doesn't return a value. Functions of this type are called for their side effects, such as printing to the console, modifying global state, or performing an action like closing a file. Here's a simple example:

```
void printMessage() {
    std::cout << "Hello, world!" << std::endl;
}
```

Listing 4.4: Using void as a function return type

In this function, a message is printed to the console. It does not return a value or reference to a value.

In Listing 4.1 we ask the user to enter two numbers that are then summed. If this has to be done multiple times in your program, this may lead to duplicate code. Instead, it would make more sense to group these statements into a function (a "labelled block" of code) that can be called from other parts of your program.

Here is a function achieving the required outcome:

```
#include <iostream>
using namespace std;

/*
 A function to calculate the sum of two user provided numbers.
*/
void calculateSum() {
    int num1, num2 answer = 1;

    cout << "Enter first number: ";
    cin >> num1;
    cout << "Enter second number: ";
    cin >> num2;

    answer = num1 + num2;

    cout << answer << endl;
}

int main() {
    calculateSum();
    return 0;
}
```

Listing 4.5: Simple Void Function Without Parameters

In this C++ example, `calculateSum` is a void function that asks a user to input two numbers and displays the sum of the numbers. Now each time you wish to determine the sum of two user entered values you can call `calculateSum` instead of duplicating code.

Question 4.1
What is displayed on the screen if the example program is run with 5 and 7 as user input?

Question 4.2
Write a void function that asks a user to input two numbers and displays the product of the numbers.

4.5.2 Variable Scoping

Did you notice in `calculateSum` there are three variable declarations (`num1`, `num2` and `answer`). These variables are *local* to the function and cannot be accessed or used by any other function or the main function.

This means trying to print out answer in the main function after the function has been called would be invalid.

```
1  #include <iostream>
2  using namespace std;
3
4  // Global variable
5  int i;
6
7  // Function 'someFunction' declaration
8  void someFunction() {
9      int j = 1; // Local variables to the 'someFunction' function
10     cout << j << endl;
11     cout << i << endl;
12 }
13
14 int main() {
15     i = 0;
16     cout << i << endl
17     someFunction();
18     return 0;
19 }
```

Listing 4.6: Variable scopes

In the above example, the integer `i` is a **global variable**. It can be accessed (and changed) in `someFunction` as well as in the main function.

Global variables are generally considered bad programming practice for a multitude of reasons. Global variables can be accessed and modified from anywhere in your program, making it hard to track down where a change is made. This can lead to code that's difficult to understand, read and debug. Functions that depend on global variables are also less reusable, since they rely on the setup of global variables even when they are used in a different context.

Integer `j` is a **local variable**. It is local to `someFunction` and cannot be accessed from anywhere else in the program, including the main function.

Question 4.3
What is the scope of each of the variables in the following program?

```
#include <iostream>

using namespace std;

int var1  = 10;

void functionA() {
    int var2 = 5;
    cout << "Inside functionA, var1: " << var1 << endl;
```

```
    cout << "Inside functionA, var2: " << var2 << endl;
}

void functionB() {
    int var3 = 15;
    cout << "Inside functionB, var1: " << var1 << endl;
    cout << "Inside functionB, var3: " << var3 << endl;
}

int main() {
    cout << "Inside main, var1: " << var1 << endl;
    functionA();
    functionB();
    return 0;
}
```

Question 4.4
Will the following code compile? Why, or why not?

```
#include <iostream>

using namespace std;

int var1  = 10;

void functionA() {
    int var2 = 5;
    cout << "Inside functionA, var1: " << var1 << endl;
    cout << "Inside functionA, var2: " << var2 << endl;
}

void functionB() {
    int var3 = 15;
    cout << "Inside functionB, var1: " << var1 << endl;
    cout << "Inside functionB, var3: " << var3 << endl;
    cout << "Inside functionB, var2: " << var2 << endl;
}

int main() {
    cout << "Inside main, var1: " << var1 << endl;
    functionA();
    functionB();
    cout << "Inside main, var2: " << var2 << endl;
    cout << "Inside main, var3: " << var3 << endl;
    return 0;
}
```

4.5.3 Functions with parameters

Functions are extremely useful, but can be limiting in the way they have been defined up till now. It would be handy to be able to send the two numbers to the `calculateSum` function instead of asking the user to type it in, in the function. To do this, functions can be extended to include parameters. The parameter of a function can be seen as values that are passed to a function.

The syntax for the declaration of a void function accepting parameters is given by:

```
void <functionName> ( <FormalParameterList> ) {
    <FunctionBody>;
}
```

The syntax of a function call is given by:

```
<functionName>(<ActualParameterList>);
```

When we define a function, we specify *formal parameters* (or formal arguments) in its declaration. These parameters are like placeholders for the values that the function will operate on. When the function is called, *actual parameters* (or actual arguments) are the specific values or variables passed into the function. The process of mapping actual parameters to formal parameters occurs when the function is called, enabling the function's logic to operate on the given inputs. Each parameter in the formal parameter list is defined by its type and unique name.

You can rewrite the `calculateSum` function to accept two numbers as parameters:

```
#include <iostream>
using namespace std;

/*
    A function to calculate the sum of two numbers.
    The function accepts two integer parameters to be added.
*/
void calculateSum(int num1, int num2) {
    int answer = num1 + num2;
    cout << answer << endl;
}

int main() {
    calculateSum(4,5);
    int a = 5;
    int b = 3;
    calculateSum(a,b);
    return 0;
}
```

Listing 4.7: Function With Parameters

Consider the `calculateSum` function defined to accept two numbers as parameters. This function demonstrates how formal parameters (`num1` and `num2`) are placeholders for the

actual values passed in during the function call.

When calling a function, the actual parameters can be either direct values (e.g., `4` and `5` in `calculateSum(4,5);`) or variables that hold values (e.g., `a` and `b` in `calculateSum(a,b);`).

Question 4.5
What is displayed on the screen if the example above is run?

Question 4.6
Write a new function called sumFloats that will accept two float values as input and display their sum.

4.5.4 Functions that return values

Sometimes, we don't just want to display the results of our calculations directly from a function. There are occasions when we might need the outcome of these calculations for further use elsewhere in our program. We have already seen that in C++, functions can perform tasks or operations. In C++, functions can also return values to the place where they were called. This feature allows functions to not only carry out tasks but also send their results back for use in other parts of your program.

For a function to return a value, it must be defined with a return type other than `void`. You're already familiar with the `int main()` function, which returns an integer (`return 0;` signals successful program completion).

Here's how you can define a function that calculates and returns the sum of two numbers:

```
#include <iostream>
using namespace std;

/*
    This function calculates the sum of two numbers.
     It accepts two integers as parameters and returns their sum
*/
int calculateSum(int num1, int num2) {
    int answer = num1 + num2;
    return answer;
}

int main() {
    int result1 = calculateSum(4,5);
    cout << result1 << endl;
    int a = 5;
    int b = 3;
    int result2 = calculateSum(a,b);
    cout << result2 << endl;
    return 0;
```

```
}
```

Listing 4.8: Function With Parameters And Return Type

In the example above, `calculateSum` is defined to return an integer. It's important to specify the exact return type the function will produce. Remember, for functions that are supposed to return a value (anything other than void), every possible path through the function must end with a return statement providing a value of the declared type. If not, you'll run into a compiler error[1].

The adjusted structure of a function with a resturn type is given by:

```
<ReturnType> <functionName> ( <FormalParameterList> ) {
    <FunctionBodyStatements>;
    return <ValueOfReturnType>
}
```

The adjusted structure of a function call is given by

```
<functionName>(<ActualParameterList>);
```

or

```
<type> <variableName> = <functionName>(<ActualParameterList>);
```

Question 4.7
Write a function named `calculateAreaOfCircle` that returns the area of a circle. The function should accept one float parameter for the radius of the circle. Use the formula $area = \pi \times radius^2$ and return the result as a float. Assume $\pi = 3.14159$.

Question 4.8
Create a function called `convertFahrenheitToCelsius` that returns the Celsius equivalent of a Fahrenheit temperature. The function should accept one float parameter for the temperature in Fahrenheit and return the temperature converted to Celsius. Use the formula $Celsius = (Fahrenheit - 32) \times \frac{5}{9}$.

Question 4.9
Design a function named `calculateAverage` that computes the average of two numbers. The function should take two integer parameters and return their average as a float.

Question 4.10
Develop a function named `calculateDistanceTravelled` that calculates the distance travelled given speed and time. The function should accept two float parameters: speed in meters per second (m/s) and time in seconds (s). It should return the distance as a float, using the formula $distance = speed \times time$.

4.5.5 General format for a function

We have considered examples of functions with and without return types and parameter lists. A general syntax for a function, that captures all combinations of return types and

[1] We will revisit this when we consider selection statements in Study Unit 5

parameters, is given by [2]:

```
<ReturnType> <functionName>([ParameterList]) {
    <FunctionBody>     // Implementation of function's operations
    [ReturnStatement] // Included if the returnType is not void
}
```

Listing 4.9: General function format

1. `ReturnType`: The return type specifies the type of data the function will return to the caller. It can be any valid data type, including `void` if the function does not return a value.

2. `functionName`: This is the identifier used to call a function. Function names should follow the naming conventions of C++ and should describe what the function is doing.

3. `ParameterList`: The parameter list specifies the input that the function accepts. Each parameter must have a type and name. There can be multiple parameters separated by commas. A function can also take no parameters (indicated by an empty list). These are the formal parameters. They are placeholders for the values being passed in. The format for the formal parameter list is given by:
   `[<parameterType1> <parameterName1> [, <parameterType2> <parameterName2>], ...]`

4. `FunctionBody`: The statements between the {} that defines the functions behaviour. This is where the algorithm/operations are implemented.

5. `ReturnStatement`: The return statement is used to exit the function and return control to the calling function. The return statement will also send a value back if the function is not void (i.e. the function has a return type). You have already seen an example of this in the main function: `return 0;`.

4.6 Separation of concerns

Separation of concerns is a design principle that keeps your code organised and readable by ensuring that each part of your code handles a specific task. When you break down a complex problem into manageable parts, it makes your solution easier to understand and adapt.

To date we have placed all code in the main function. As programs become more complex, understanding what the code is doing becomes unmanageable. We will consider ways of making the code more manageable and maintainable by removing repeating code that serves a single function into functions. These functions can be defined in the same file as the main function (discussed in Section 4.6.1) or in separate files (refer to Section 4.6.2).

[2]The notation used to specify the general syntax of C++ is as follows. When a syntax element is specified in <>, the syntax element is mandatory. Syntax elements in [], are optional for the base case and are specified when needed.

4.6.1 In a single file

You can apply the separation of concern principle in a single file by making use of functions. Each function should be responsible for one specific task.

In C++, the compiler needs to know about a function before you can use it. This is so it can check that you are correctly using the function, such as giving it the correct number of arguments and handling its return value properly.

We will consider the two placements for the function implementation of the funtion, that is before the `main` function and after the `main` function.

4.6.1.1 Above the main

You can ensure that the compiler knows about your functions by writing all your function definitions at the start, before you use them. This way, the compiler already knows everything it needs to about a function to make sure that the function call is valid.

```
#include <iostream>

using namespace std;

void printPersonalisedHelloWorld(string name) {
    cout << "Hello " << name << endl;
}

string getName(){
    string name;
    cout << "Please enter your name: ";
    getline(cin, name); // Use getline to read a line of text
    return name;
}


int main() {
    string name = getName();
    printPersonalisedHelloWorld(name);
}
```

Listing 4.10: Declaration of Functions above the Main

Making sure that the function definitions are above the `main` function where they are called allows the compiler to check that the calls from the main are valid. By the time the compiler gets to `main` it has already processed the necessary information about each function (name, parameters and return type).

When functions make calls to one another, you will have to ensure that their order is correct. The compiler needs to know about a function before it's called by any other function (not just the `main`).

```
#include <iostream>
using namespace std;

double getPi(){
    return 3.14159;
}

double calculatePerimeter(double radius) {
    return 2 * getPi() * radius;
}

double getRadiusSquared(double radius) {
    return radius * radius;
}

double calculateArea(double radius) {
    return getPi() * getRadiusSquared(radius);
}

int main() {
    double radius;
    cout << "Enter the radius of the circle: ";
    cin >> radius;

    cout << "Area of the circle: " << calculateArea(radius) << endl;
    cout << "Perimeter of the circle: " << calculatePerimeter(radius) <
}
```

Listing 4.11: Declaration of Multiple Functions above the Main

In this example `getPi` needs to be declared before it is used in `calculatePerimeter`. Similarly, both `getRadiusSquared` and `getPi` need to be defined before `calculateArea` uses them.

But if you do it this way, your code can become cluttered. Your main function ends up being at the bottom of your code, which can make it harder to read. C++ offers a better way to tell the compiler about your functions ahead of time and placing the implementation of the function after the `main` function.

4.6.1.2 Forward declarations

A function declaration (or prototype) gives the compiler all the information it needs to check that the function is being used correctly without implementation details. A declaration includes the function's return type, name and parameter types (if any). The body of the function, i.e. the code that will be run when the function is called, is not included in the declaration. Now instead of needing the entire function with body before any calls to the function, the declaration alone serves as a "promise" to the compiler that a function like this exists and is implemented somewhere else. This is called a forward declaration.

```
int add(int, int);
```

Listing 4.12: Function declaration

In this snippet, the declaration includes the function's return type (int), its name (add), and its parameters (two integers). Note, it is not necessary to name the parameters in the declaration. With this information, the compiler understands that an `add` function exists, expects two integer arguments during any call, and will return an integer value. This setup also guarantees that any return value from `add` will be managed appropriately. The function definition (body) can be provided later in the code.

```
#include <iostream>
using namespace std;

// Forward declarations
double getPi();
double calculatePerimeter(double radius);
double getRadiusSquared(double radius);
double calculateArea(double radius);

int main() {
    double radius;
    cout << "Enter the radius of the circle: ";
    cin >> radius;

    cout << "Area of the circle: " << calculateArea(radius) << endl;
    cout << "Perimeter of the circle: " << calculatePerimeter(radius) <
}

// Function implementations
double calculatePerimeter(double radius) {
    return 2 * getPi() * radius;
}

double calculateArea(double radius) {
    return getPi() * getRadiusSquared(radius);
}

double getPi() {
    return 3.14159;
}

double getRadiusSquared(double radius) {
    return radius * radius;
}
```

Listing 4.13: Forward Declaration of Multiple Functions

When compared to listing 4.11, the code in listing 4.13 is much easier to read and understand. With the entry point (`main`) at the top of the file, you can quickly see the program's

main flow and purpose without reading through the functions first. This decreases the clutter in the file.

The concept of forward declarations makes it easier to ensure that the compiler is aware of functions before their use. As you work on bigger projects, managing forward declarations and function implementations in a single file can become difficult. This is where the division into header (.h) and implementation (.cpp) files helps us further apply the separations of concern principle.

4.6.2 Header and implementation files

C++ allows you to separate code into header (`.h`) and implementation (`.cpp`) files. In the header files you can declare the interfaces of your functions in logically grouped files. This makes your code cleaner and less cluttered. It also enables you to share and understand code with more ease, since you can scan the interface to get an idea of the purpose of the code without having to read through 100s of lines of code.

4.6.2.1 Header Files

The header (`.h`) files contain the function declarations that can be used in other files. It serves as an interface to a set of functionalities in your program.

When you include a header file in your `.cpp` file, you make the declarations available to the compiler.

```
#ifndef AREAS_H
#define AREAS_H

// Forward declaration of functions
double calculateAreaOfCircle(double radius);
double calculateAreaOfSqaure(double length);
double calculateAreaOfRectangle(double length, double width);

#endif
```

Listing 4.14: Example Header File (areas.h)

The `ifndef`, `define`, and `endif` preprocessor commands ensure that the header file is included only once by the compiler, avoiding multiple definition errors.

You can now include `area.h` in any implementation file in order to tell the compiler that these functions exist. Before you use these functions, you need to define the body of the functions somewhere. This will be in the corresponding implementation file.

4.6.2.2 Implementation Files

The implementation (`.cpp`) files contain the actual code for the functions declared in the header file. This ensures that the actual details of the implementation can be hidden from anyone else that has to use your code.

```cpp
#include "areas.h"

double calculateAreaOfCircle(double radius){
    double pi = 3.14;
    return pi * radius * radius;
}

double calculateAreaOfSqaure(double length){
    return length * length;
}
double calculateAreaOfRectangle(double length, double width){
    return length * width;
}
```

Listing 4.15: Example Implementation File (areas.cpp)

By compiling and linking these files together, you can create a single executable.

You can now use these functions anywhere where you include the `areas.h` header file. The compiler knows about these functions because of the forward declarations in the header file.

```cpp
#include <iostream>
#include "areas.h" // Include the header file to use its declarations

using namespace std;

int main() {
    double radius = 5.0;
    double squareLength = 4.0;
    double rectangleLength = 6.0, rectangleWidth = 3.0;

    // Use the functions declared in areas.h and defined in areas.cpp
    double circleArea = calculateAreaOfCircle(radius);
    double squareArea = calculateAreaOfSqaure(squareLength);
    double rectangleArea = calculateAreaOfRectangle(rectangleLength,
                                    rectangleWidth);

    // Display the results
    cout << "Area of circle: " << circleArea << endl;
    cout << "Area of square: " << squareArea << endl;
    cout << "Area of rectangle: " << rectangleArea << endl;

    return 0;
}
```

Listing 4.16: Inluding the declarations in area.h for use in the `main`

By organising your code in this way, you make it much easier read, and to reuse parts. If you have other functions in a separate `.cpp` file that need access to the function declared in `areas.h`, you can simply include the header file.

Note the syntax of the preprocessor `include` commands for user-defined files. To distinguish between libraries defined within C++ and libraries defined by an external party framework or you the programmer, C++ libraries are specified in angle brackets without the .h extension. User-defined libraries/files are include using double inverted commas and the file extension is included. If the file resides in the same file as what is including it, you do not need to specify a path to the file. If it resides in a different folder, you will need to currently specify where it is.

Separation into .h and .cpp files improves the compilation time, as changes in the implementation (`.cpp` files) do not require recompilation of the code that uses these functions, as long as the interface in the header file remains unchanged.

Question 4.11
You are tasked with programming a basic calculator that can add, substract, multiply and divide. Implement the `arithmetic.h` and `airithmetic.cpp` files with functions `add`, `substract`, `multiply` and `divide` and call them from your `main.cpp`.

Question 4.12
You are tasked with calculating the price of mowing lawns of different shapes and sizes. Prices are per square meter. Create a new set of functions in `lawn.cpp` (which you forward declare in `lawn.h`) that can calculate the price charged for a lawn. There should be 3 functions:

1. `calculateCircularLawnPrice` which accepts the price per square meter and the radius of the lawn as parameters (doubles) and returns the total price to mow the lawn.

2. `calcualteSquareLawnPrice` which accepts the price per square meter and the length of the lawn as parameters (doubles) and returns the total price to mow the lawn.

3. `calcualteRecctangularLawnPrice` which accepts the price per square meter, the length and breadth of the lawn as parameters (doubles) and returns the total price to mow the lawn.

Do not recalculate the areas. Instead, use the `areas` functions provided above in the `areas.h` and `areas.cpp` files.

Test these functions in your main.

In order to use header (`.h`) and implementation (`.cpp`) files, you need to compile and link them correctly. More details are given in the Appendix.

4.7 Explicit Type Conversions

Type conversions were introduced in Study Unit 3 where the focus was on implicit type conversions. Remember, implicit type conversions are done by the compiler. For example, when a value of one type is being assigned to a variable of another type. In the example below, an implicit conversion from int to double.

```
int num_int = 10;
double num_double = num_int;
```

Explicit type conversions requires the programmer to tell the compiler to convert one type to another. This is useful when there's a need to convert between incompatible types or when precision loss is acceptable.

Possible scenarios where explicit type conversion is needed

- Consider a return type of float and two int's as input for a division function. If the function divides to the 2 ints, then integer division is applied. Change one of the input parameters to a float to ensure floating point division
- When a return type is not compatable with the type of the variable to which the return value of the function is assigned.

The use of explicit type conversions provides more control and clarity over the code but may also lead to potential issues such as data loss or undefined behaviour if used improperly. Therefore, it's important to use them judiciously and understand their implications.

So when do we use explicit type conversion

- Converting Between Primitive Types
- Enforcing Type Conversions
- Avoiding Implicit Conversions
- Casting Pointers (later)
- Casting Away Constness (later)

Explicit type conversion can be done using **C-style casting**, **static_cast**, *functional notation*, *dynamic_cast*, *const_cast*, and *reinterpret_cast* operators. In this module we will consider the first two, listed in **bold**.

- C-style casting: Explicit conversion from double to int using C-style casting

```
double num_double = 3.14;
int num_int = (int)num_double;
```

- `static_cast`: Explicit conversion from double to int using `static_cast`

```
double num_double = 3.14;
int num_int = static_cast<int>(num_double);
```

Other casts `dynamic_cast`, `const_cast`, and `reinterpret_cast`, listed in *italics*, are used for specific scenarios like casting pointers, dealing with inheritance, etc. The functional notation is available in later standards of C++.

4.8 Function overloading

Sometimes we need to perform the same task with different parameters. For example, think of mathematical operations that can be applied to different types of numbers or a different number of arguments. Consider a function `multiply` that calculates the product of numbers. The "purpose" stays the same, regardless of whether you are multiplying integers or doubles, or even more than two numbers.

Function overloading allows you to write multiple versions of the same function (same name) with different parameter lists. The parameter list can differ based on the types or number of parameters. The function's name together with it's parameters (number and type) is also known as the function's signature. The compiler distinguishes between these overloaded functions based on their signature. When an overloaded function is called, the compiler determines the most appropriate version to use by matching the call's argument types with the parameter types of the overloaded functions.

Imagine you need to write a program to inspect variables of unknown types and perform actions based on their types. You decide to implement a set of overloaded functions named `printType` to identify and print the type of the variable passed to it.

```
#include <iostream>
#include <string>

using namespace std;
// Overload for int type
void printType(int) {
    cout << "Type: int" << endl;
}

// Overload for double type
void printType(double) {
    cout << "Type: double" << endl;
}

// Overload for char type
void printType(char) {
    cout << "Type: char" << endl;
}

int main() {
    int integerVar = 10;
    double doubleVar = 10.5;
    char charVar = 'A';

    printType(integerVar);
    printType(doubleVar);
    printType(charVar);

    return 0;
```

```
}
```

Listing 4.17: Example of Function Overloading

In this example, when you call `printType` with a variable, the C++ compiler selects the appropriate overload based on the argument's type.

4.8.1 Default Parameters

Default parameters in C++ provide a way to specify default values for function parameters. This allows you to call the function with fewer arguments than it has parameters. The default values will be used for the parameters that have not explicitly been given a value in the call.

You can specify default values for one or more parameters of a function in the function declaration. When the function is called and those parameter arguments are left out the default values are automatically used. Default values have to be assigned from right to left. In other words if you have a function with three parameters, you cannot only assign a default value to the middle parameter.

```
#include <iostream>
using namespace std;
// Function with a default parameter
void printGreeting(string name = "World") {
    cout << "Hello, " << name << "!" << endl;
}

int main() {
    printGreeting();              // Prints "Hello, World!"
    printGreeting("Alice");       // Prints "Hello, Alice!"

    return 0;
}
```

Listing 4.18: Example of Default Parameters

In this example, the `printGreeting` function is defined with one parameter, name, which has a default value of "World". When `printGreeting` is called without an argument, it uses the default value and prints "Hello, World!". When called with a specific name, it uses that name instead of the default.

4.8.2 Ambiguous function signatures

Function overloading in C++ allows you to have multiple functions with the same name but different parameter lists. Default parameters enable you to define a function with one or more parameters that have default values. However, combining these two features can sometimes lead to ambiguity, where the compiler does not know which function you are trying to call.

```
#include <iostream>
using namespace std;

void printValue(int a) {
    cout << a << endl;
}

void printValue(int a, int b = 0) {
    cout << a << endl;
    cout << b << endl;
}

int main() {
    int x = 10;
    int y;
    printValue(x); // Ambiguous call
    return 0;
}
```

Listing 4.19: Ambiguous Function Calls

The ambiguity arises because the compiler cannot determine which version of the `printValue` function you're trying to call. Both versions could potentially match the call `printValue(x)`. The first function matches directly because it expects one argument. The second could also match because it has a default value for the second parameter, making the second parameter optional. The compiler will throw an error that the call to 'printValue' is ambiguous.

Note: When the function is called with two arguments, there is no ambiguity because only the second overloaded function can take two parameters. Thus the call `printValue(2,3)` is not ambiguous and will not cause a compiler error.

4.9 Documenting Functions

When writing functions, and especially more complex functions, it is advisable to document the functions for your reference at a later point in time or for someone else who is using your function so that they understand what the function does, the parameter and return types of the function as well as the limitation of the function.

There are tools that can be used to write documentation for your code that automatically generate a set of documentation for your code. As we are not using all the functionality provided by these tools, we will describe the function definition (Function Descriptions) and restrictions placed on and by the function (Pre- and Post-conditions). The documentation of the functions will be placed in the header file where the functions are defined. This is typically the file that will be provided to a programmer when they are using a library.

4.9.1 Function Descriptions

Function descriptions will include a:

- short textual *description* of the function
- description of the *formal parameter list*
- description of the *return type*

To ensure that all functions are documented in the same manner, the following comment should be placed before each function defined in the header file.

```
/**
A short description of the functionality the function provides
Parameters: A list of the types of the parameters and what each
    parameter is used for, if not obvious.
Return type: The type being returned and a short description of
    the meaning of the type.
**/
```

For example, for the `pintValue(int)` in Listing 4.19, assuming that the parameter may not be negative, the description of the function may read as follows:

```
/**
Prints an integer value to the console.
Parameters: A single integer parameter
Return type: none (void)
**/
```

In this example, the parameter could be written in more formal manner, for example: **int** a, where a is written to the console.

What is important is to remain consistent for all descriptions within at least the (header) file in which the function is being defined. We will use function pre- and post-conditions to provide bounds for parameters and return types.

4.9.2 Pre- and Post-conditions

Before calling the function, certain pre-conditions must be satisfied to ensure the function's proper execution. These pre-conditions primarily pertain to the types and values of the parameters accepted by the function. Failing to meet these pre-conditions may lead to erroneous behaviour or even runtime errors.

Once the function executes, it produces certain post-conditions that define the net result of its operation. Understanding these post-conditions is crucial for interpreting the outcome of the function's execution and for utilising its results effectively.

In summary,

- *pre-conditions*, what must be true before the function is called. This is especially true pertaining to the types/values of the parameters. For example, for division, if 2 values are accepted by the function, e.g. x and y, then the second value (y) may not be 0 if the function is calculating x/y

- *post-conditions*, what is the net result of the function. This may include the meaning of the value returned or bounds of the return type.

Pre-conditions and post-conditions are included in the function description, for example:

```
/**
Prints an integer value to the console.
Parameters: A single integer parameter
Return type: none (void)

Pre-condition: The integer parameter must be >= 0
Post-condition: The value of a is printed to the console
**/
```

A more concise way of specifying the pre-condition is to state that
For parameter **int** a, where a >= 0.

Later, pre-conditions and post-conditions, along with the function invariant can be used to determine the correctness of the function. This is beyond the scope of these study units.

4.10 Document Change Log

Date	**Change**
18 Mar 2024	Initial preparation and presentation of the document.
14 Apr 2024	Addition of default parameters, function overloading and pre and post conditions.

A Compiling and Linking with Makefiles

Makefiles are data files that can be used by the make program to intelligently compile and link a system consisting of multiple C++ source files. It is the responsibility of the designer of the system to specify the detail needed by the make program to successfully compile and link the system. In order to be able to write your own makefiles, you need to understand how systems are compiled and linked without the aid of the make program. This is discussed in Section A.1. In Section A.2 we discuss how the make program automates this process. The content and structure of makefile entries are explained. In Section A.5 we discuss how macros and rules can be used to write generic makefile entries. Understanding this enables you to write smaller and versatile makefiles that can easily be adapted to describe other systems. Finally we conclude with two small sections to mention common errors that are made by novices when creating there own makefiles and to introduce a few challenges to students who like to deepen their understanding through practical experience.

A.1 Command line compiling and linking

A.1.1 Compiling of a single C++ file

We assume that you use a text editor such as SciTE to write C++ programs and use the console terminal program of a linux operating system to type command-line instructions to compile and run these programs. When writing a C++ program, the source code has to be compiled before it can be executed. When the whole system is included in a single .cpp file, the process is trivial. You simply have to invoke the GCC compiler with the g++ command and specify the .cpp file name as input file. For example if your source code file is named **HelloWorld.cpp**, you can compile the program with the following command:

```
g++ HelloWorld.cpp
```

This command will create the executable file with the default name (a.out). This program can then be executed with the following command:

```
./a.out
```

Although only a single command is issued to perform this task, the following three steps are automatically executed to obtain the final executable program:

1. Compiler stage: All C++ language code in the .cpp file is converted into a lower-level language called Assembly language;
2. Assembler stage: The assembly language code made by the previous stage is then converted into object code which are fragments of code which the computer understands directly. An object code file is normally stored with .o as its file extension.
3. Linker stage: The final stage in compiling a program involves linking the object code to code libraries which contain certain "built-in" functions. This stage produces an executable program, which is named a.out by default.

It is possible to specify the output file name by using the **-o** option followed by the name of the output file. For example if your source code file is named **HelloWorld.cpp**, you can compile the program with the following command:

```
g++ HelloWorld.cpp -o HelloWorld
```

This command will create the executable file with the specified name **HelloWorld**. This program can then be executed with the following command:

```
./HelloWorld
```

Note that you can call the executable whatever you want. It need not have the same name as the .cpp file.

A.1.2 Header files

When writing code in terms of classes (using the object oriented programming paradigm), it is customary to save the code of a given class in two files:

1. Header file: This file commonly contains forward declarations (prototypes / signatures) of classes, member functions, instance variables, and other identifiers. A header file is normally saved with .h as its file extension.
2. Implementation : This file contains the complete definitions of the member functions that are listed in the header file. The implementation file is normally saved with .cpp as its file extension.

When the code for such class needs to be compiled, it has to be made into a single document to be compiled by the compiler. This is done by using a compiler directive to insert the header file in the .cpp file before compiling the combined content of the two files. This is specified by using the a compiler **#include** command at an appropriate position in the implementation file. For example if a header file is named **Account.h**, it can be included in any .cpp file that make use of the classes, functions or variables declared in it by simply adding the following line of code in the .cpp file:

```
#include "Account.h"
```

When a .cpp file is compiled, the code in all the the header files inlcuded in it is also compiled. For example if a file named **Account.cpp** includes the file **Account.h** with the above compiler directive, the constructed file that is compiled by the following command consists of the code contained in both these files:

```
g++ Account.cpp -o Account
```

Multiple header files can be included in a single .cpp file. A single header file may also be included in multiple .cpp files.

To avoid multiple declarations when a given header file is included in multiple .cpp files in the same compilation, it is customary to guard the content of header files as a defined block (with a unique name). This guard compiles the inclusion only on condition that the defined block was not perviously encountered during the current compilation. The following illustrates how the content of a header file can be guarded:

```
#ifndef H_GUARD
#define H_GUARD

/* The forward declarations of classes, functions,
 variables, and other identifiers comes here */

#endif
```

The content of the header file is given a unique name with the command `#define H_GUARD`. To simplify naming, this name is usually chosen to correlate with the header file name.

The content of the file is placed in a conditional block starting with `#ifndef H_GUARD` and ending with `#endif`. This conditional statement specifies that the body of this statement should only be considered if the given name is not defined. Because the specified name is defined inside this block, the content of the block will be compiled only the first time it is encountered during the compilation process.

A.1.3 Compiling multiple source files

When your program becomes very large, it makes sense to divide your source code into separate easily-manageable .cpp files. It can be compiled issuing a single command. For example, if a system is made up of two .cpp files named **Bike.cpp** and **Tricycle.cpp** respectively and a single common .h file named **Wheel.h**. The command to compile all, assuming **Wheel.h** is properly included in both .cpp files, is as follows:

```
g++ Bike.cpp Tricycle.cpp
```

Note that the first two steps taken in compiling the files are identical to the previous procedure for a single .cpp file. However, the two compiled files are linked together at the Linker stage to create one executable program. Because the name of the output file was not specified, the output file will be named **a.out** in this case.

A.1.4 Compiling without linking

The steps taken in creating the executable program can be divided into steps, separating the compiling process from the linking process. Firstly all .cpp files can be compiled and stored as .o files and in a final step the .o files can be linked to create the executable program.

You can use the -c option with g++ to create the corresponding object (.o) file from a .cpp file. For example, the command to compile both the above mentioned .cpp files, without linking them is the following:

```
g++ -c Bike.cpp Tricycle.cpp
```

When executing this command, the compiler will stop after the assembler stage. The object code is written to disk in two .o files corresponding with the .cpp files.

A.1.5 Linking compiled code

The compiler can use either .cpp or .o files when issued (*without* the -c option) to create the required executable file. The following command can thus be issued to link the compiled code that was created in the previous section:

```
g++ Bike.o Tricycle.o
```

If you would like to name the executable file something else than **a.out**, you can specify it with the -o option. For example if you would like to name the executable file of the above mentioned system **GoRide**, you can issue the following command:

```
g++ Bike.o Tricycle.o -o GoRide
```

Suppose you have written a system consisting of the following files

C++ file	**Includes**
Bike.cpp	Wheel.h, Bike.h
Tricycle.cpp	Wheel.h, Tricycle.h
main.cpp	Bike.h, Tricycle.h

When issuing the following commands, the intermediate .o files for this system will be created on disk, wherafter they are linked and a single executable file called **GoRide**[3] is created.

```
g++ -c Bike.cpp Tricycle.cpp main.cpp
g++ Bike.o Tricycle.o main.o -o GoRide
```

A.1.6 Re-compiling after a small change

If you have changed one of the source files in the above mentioned system it is not necessary to re-compile all. Only the files that are changed as a result of changing a source file need to be recompiled. For example, suppose only the Bike.cpp was changed. Knowing how the system is designed, you will realise that **Tricycle.o** and **main.o** are not affected by this change. A complete re-build of **GoRide** incorporating the change can thus be achieved by issuing the following commands:

```
g++ -c Bike.cpp
g++ Bike.o Tricycle.o main.o -o GoRide
```

Similarly if you have changed **Tricycle.h** the re-build would require the following commands:

```
g++ -c Tricycle.cpp main.cpp
g++ Bike.o Tricycle.o main.o -o GoRide
```

We have to admit that in this small system it would not make a big difference if you omit the creation of one or two .o files. However, when developing very large systems containing scores, or even hundreds, of files, it often saves a lot of time if only the appropriate files are updated.

[3]Note that the names given to files are arbitrary and should be chosen to be descriptive of your system

A.1.7 Compiler flags

When compiling you can issue the compile command with options. One of the options (`-o`) was used in the example in Section A.1.1 to specify the output file. A host of other options, known as compiler flags, are available for most compilers. However, they are not standardised for all compilers. Table 1 lists some useful flags that are available for the GCC compiler that is used on campus.

Table 1: Useful g++ flags

Flag	**Usage**
-o	To specify the output filename
-w	Disable all warning messages. Note that the the use of -w to suppress the compiler warnings is against our coding standards, because for better reliability one should take compiler warnings serious, and not ignore them as if they don't exist!
-Wall	Enable most compiler warnings
-Werror	Treat compiler warnings as errors Note that the use of this flag really show that you are serious about compiler warnings because you actually want to turn them into errors.
-pedantic	Issue all the warnings demanded by ISO C++; reject all programs that use language extensions supported by the GCC compiler that is not part of the ISO specification of the language.
-pedantic-errors	Like `-pedantic`, except that errors are produced rather than warnings.
-g	turns on debugging. This makes your code ready to run under gdb.
-O	turns on optimization, you may also specify levels (-O2).
-E	outputs the preproccessor output to the screen (stdout).
-static	On systems that support dynamic linking, this prevents dynamic linking with the shared libraries
-c	compiles the given source down to an object file. This is used projects that have multiple files to reduce compile time.
-MM	outputs the Makefile dependancies for the source file(s) listed.

Any number of these flags can be inserted in the compile command. For example, the following command will compile the `HelloWorld.cpp` program that was mentioned in Section A.1.1 to create an executable file called `MyWorld`. Furthermore, it will treat all warnings as errors and also include additional debugging information in this executable.

```
g++ -Werror -g HelloWorld.cpp -o MyWorld
```

A.1.8 Wild card characters

When specifying file names (or paths), the asterisk character ***** can be used as a substitute for zero or more unspecified characters. Similarly the question mark **?** can be used as a substitute for one unspecified character. Ranges of characters enclosed in square brackets ([and]) serve as a substitute for any of the characters in the specified ranges; for example, [A-Za-z] substitutes any single capitalized or lowercase letter. The following command is an example that shows how these substitues can be used to write more generic commands. This command compiles all the .cpp files in the current directory that has a lower or upercase **k** as the third character in the filename:

```
g++ ??[kK]*.cpp
```

By using naming conventions, for example starting a specific subset of filenames with a common character, can enable you to refer to these files in terms of a wild card pattern.

A.2 Automating the build process

A program called **make** can be used to build source codes automatically. Apart from automating the build process, this program was designed to only build source code that has been changed since the last build. The make program gets the inter-dependency of the source files in the system from a text file called **Makefile** or **makefile**. If no path is specified it is assumed that this file resides in the same directory as the source files.

Make checks the modification times of the files, and whenever a file becomes *newer* than something that depends on it, it runs the build script accordingly.

A.3 Entries in a makefile

Each entry in the Makefile uses the following format:

target: source file(s)
→ *command*
...

A target given in such instruction is a file which will be created or updated when any of its source files are modified. The list of source files is also called the dependency list of the entry. The dependency list is a list of file names separated by spaces. One or more commands can be listed. The command(s) given in the subsequent line(s) are executed in order to create the target file. The → in the above format represent a **tab character**. **Each command must be preceded by** →. The → is used by **make** to distinguish between commands and dependency rules.

The **make** program uses the entries in the makefile to determine which command(s) should be issued to update the system. The commands that are executed are determined by the files that have changed since the last build. For example, if **Bike.cpp**, in the example described in Section A.1.5, is changed it becomes newer than **Bike.o** that depends on it. The make program must then issue a command to create a new **Bike.o**.

The information needed by **make** to know when **Bike.o** needs to be updated, and what command should be executed to update **Bike.o** is listed as follows:

```
Bike.o:  Bike.cpp Bike.h Wheel.h
     g++ -c Bike.cpp
```

The above entry states that whenever **Bike.o** is *older* than **Bike.cpp, Bike.h** or **Wheel.h**, the command `g++ -c Bike.cpp` should be issued.

As a result of execution of the above mentioned command, **Bike.o** then becomes *newer* than **GoRide**, that in turn depends on **Bike.o**. Once again the **make** program must issue a command to create a new version of **GoRide**. The files on which **GoRide** depend, and the command to be issued when any of these files are updated is listed in the makefile as follows:

```
GoRide:  Bike.o Tricycle.o main.o
     g++ Bike.o Tricycle.o main.o -o GoRide
```

The following is a complete listing of an example makefile for the above mentioned system:

```
GoRide: Bike.o Tricycle.o main.o
     g++ Bike.o Tricycle.o main.o -o GoRide

Bike.o: Bike.cpp Bike.h Wheel.h
     g++ -c Bike.cpp

Tricycle.o: Tricycle.cpp Tricycle.h Wheel.h
     g++ -c Tricyle.cpp

main.o: main.cpp Bike.h Tricycle.h
     g++ -c main.cpp
```

When the make program reads this data, it will attempt to create only one specified target at a time. If one or more of the items in the dependency list of the current target is missing or outdated, it will find the instructions to build them in subsequent entries and execute them in the order they are mentioned in the dependency list. It is important that the rule to create any item in any given dependency list is listed after all its occurrences in dependency lists.

A.3.1 Using the makefile with make

Once you have created your makefile and your corresponding source files, you are ready to use make. If you have named your makefile either Makefile or makefile, make will recognize it and use it if you issue the following command at the command prompt:

```
make
```

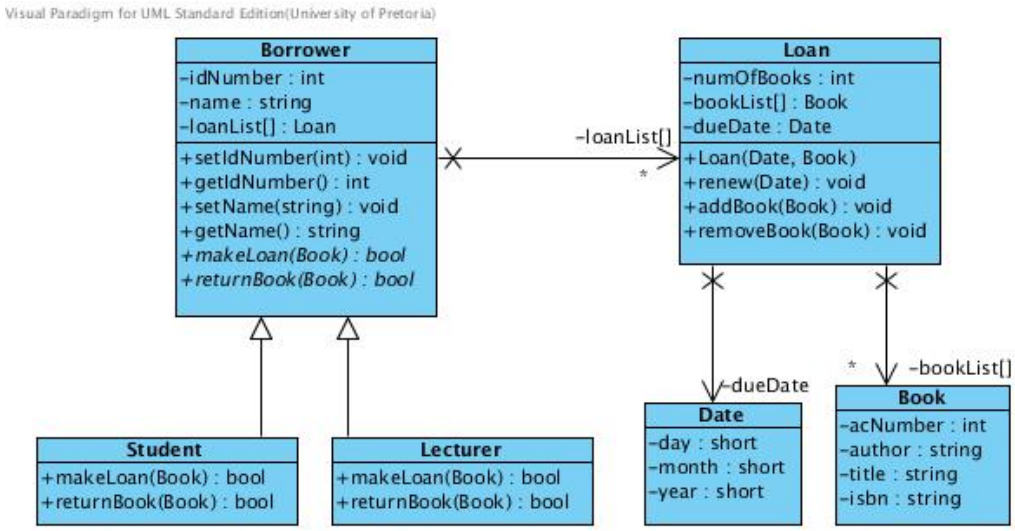

Figure 1: Borrower-Loan Class diagram

The default entry point is the top entry in the makefile. If the target of the top entry in your makefile is the final executable you wish to create you do not have to specify any parameters when issuing the make command. You can also use the make program to create a specific target by passing the required target as a command line parameter to the make program as the entry point. For example if you would like to create **Tricycle.o** using the above makefile with make you can type the following command:

```
make Tricycle.o
```

If you do not wish to call your makefile one of these names, you can give it any name you desire. If you do not use the default name, you have to specify the name of the makefile by using the -f option when invoking the make program. For example if you named your makefile **MyMakeData** you can specify that the make program must use this file with the following command:

```
make -f MyMakeData
```

A.3.2 Dependency lists

To create the .o files of a system that contains a large number of files, it is important to include all the files that each .cpp depends on, in its dependancy list. A particular .cpp file depends on its own .h file as well as all the .h files that are directly or indirectly included in the .cpp file. For example if you are given the system that contains the classes shown in the UML class diagram of figure 1, and the C++ definition of each of the classes in the diagram is stored in an .h file with the same name as the class, then the `makefile` entry to create `Student.o` should be:

```
Student.o: Student.cpp Student.h Borrower.h Book.h Loan.h Date.h
        g++ -c Student.cpp
```

It does not make sense to use wild card characters in dependency lists. It will defeat the purpose of the dependency list.

A.3.3 Continuation Line

Sometimes entries in a makefile tend to become very wide and consequently difficult to read and maintain. You are advised to break long lines into more lines. A backslash ("\") at the end of a line indicates that the next line should be interepreted as the continuation of the current line. For example is the following entry is equivalent to the one above.

```
Student.o: Student.cpp \
           Student.h  \
           Borrower.h \
           Book.h \
           Loan.h \
           Date.h
    g++ -c Student.cpp
```

It is important that the backslash that indicates that the current line continues on the next line is followed immediately by a newline. The continuation line may start at the margin, or may be indented (even with tabs). It is more readable if you use indentation on continuation lines.

A.3.4 Linking order

When having to link a number of .o files, it is important to list them in the correct order for the link command. They are always created in the order of which they are listed. For example if fileA.cpp directly or indirectly includes fileB.h, then fileB.o should appear before fileA.o in the list of .o files to link.

For example, if we assume that a C++ file containing the `main` function, called `LibSys.cpp`, use the classes shown in the UML class diagram of Figure 1, the following makefile entry will link the .o files to create an executable file called `LibSys`:

```
LibSys: Book.o Date.o Loan.o Borrower.o Student.o Lecturer.o
        g++ Book.o Date.o Loan.o Borrower.o \
            Student.o Lecturer.o LibSys.cpp -o LibSys
```

Note that `Book.o` and `Date.o` may appear in any order because thy are independent of one another. However, both of them have to be listed before Loan.o because Loan.o depends on them. Likewise, Loan.o should be before Borrower.o, which should in turn be before Student.o and Lecturer.o. The last two objects are independent of one another and may therefore be listed in any order.

Because the order in which files are named in a link command is significant, using wild card characters in a linking instruction may lead to unwanted results.

A.4 Adding custom commands

You can define any other command-line commands that you frequently use in your makefile. Do this by specifying it as a target without any dependencies and listing the command(s) that you would like to execute when you invoke the custom command below it, preceded by the usual tab characters.

The following shows the format for such custom command:

target:
→ *command*
→ *command*
...

One of the most common custom commands that are found in makefiles is the **clean** command. This command is used to issue a command to delete all interim files and other redundant files the current directory. This is sometimes useful to force a full compilation with the next issue of a make command, for example when the system needs to be recompiled on a different operating system.

On Linux a file is deleted (removed) with the **rm** command. The following is a typical definition of a custom command named **clean**:

```
clean:
     rm -f *.o *~
```

This command uses * as a *wild card* character to specify that all files with .o extension should be deleted. In the same way `*~` indicates that all files extension ending with `~` are also included in the list of files to be deleted. These are typical automatic backup files. The -f option supresses the error message that might be generated, for example when the command is issued while no such file exists.

Custom commands are executed by passing the required command as a command line parameter to the make program. For example if you would like to execute the above **clean** command, you can type the following command:

```
make clean
```

Sometimes when you recompile after a change you may see the message that states that your target is up to date, while you know it is not. In such case you can force a full compilation by issuing the clean command. If you often find that you need to use this "clean" option, it is likely that some of your dependancies are not included. When not porting to a differnt operating the use of "clean" defeats the purpose of the makefile. If all the file dependencies are correct in the makefile, you should never need the "clean" unless you port to another operating system.

Custom commands are powerful tools. It not only enables you to code some frequently used actions like removing backups or making tarballs to your makefile, it also enables you to add additional steps to the compilation process. For example if you have an application called **dingamaging** that takes a **.dmg** file to generates a **.cpp** file, the step to generate the .cpp file can also be included in the makefile.

Back to the Bicycle example. Suppose you were using this fictitious application to develop the code. The source code will then be in two files called `Bike.dmg` `Tricycle.dmg` respectively. You will need to apply **dingamaging** to these files to create `Bike.cpp` and `Tricycle.cpp`. The following is a complete listing of an example makefile for this system if it were developed with the help of a third party application called **dingamaging**:

```
GoRide: Bike.o Tricycle.o main.o
  g++ Bike.o Tricycle.o main.o -o GoRide

Bike.o: Bike.cpp Bike.h Wheel.h
  g++ -c Bike.cpp

Tricycle.o: Tricycle.cpp Tricycle.h Wheel.h
  g++ -c Tricyle.cpp

main.o: main.cpp Bike.h Tricycle.h
  g++ -c main.cpp

Bike.cpp: Bike.dmg
  dingamaging Bike.dmg

Tricycle.cpp: Tricycle.dmg
  dingamaging Tricyle.dmg
```

A.4.1 Comments

As with any source, the 'source' of the makefile can, and should, also be commented to explain it to possible readers. Any text preceded by the # character is ignored by the make program. Therefore, you can include complete comment lines by starting a line with #, or comments to the right of a statement in the makefile. The following listing of our example makefile include some comments:

```
# Linking the object code of the complete system:
GoRide: Bike.o Tricycle.o main.o
  g++ Bike.o Tricycle.o main.o -o GoRide

# Commands for partial compilation of c++ source files:
Bike.o: Bike.cpp Bike.h Wheel.h
  g++ -c Bike.cpp

Tricycle.o: Tricycle.cpp Tricycle.h Wheel.h
  g++ -c Tricyle.cpp

main.o: main.cpp Bike.h Tricycle.h
  g++ -c main.cpp

# Custom command:
clean:
  rm -f GoRide *.o *~ # deleting executable, .o's and backups
```

A.5 Reducing the size of a makefile

A.6 Introduction

The make program has many features. Two of the powerful features offered are the ability to define macros and the ability to specify generic rules by using some built-in macro's. These features combined with the use of wild card charactres (see Section A.1.8) enables the programmer to write more concise makefiles.

A.6.1 Macros

The make program allows you to use macros, which are similar to variables, to store names of files. For example you can define a name for the list of object files as follows:

```
OBJECTS = Bike.o Tricycle.o main.o
```

The make program automatically expand a macro when it runs. Whenever the macro name appears within round brackets and preceded by a dollar sign, it will be replaced by its defined content. The following is a listing of our sample Makefile again, using the above mentioned macro.

```
OBJECTS = Bike.o Tricycle.o main.o

# Linking the object code of the complete system:
GoRide: $(OBJECTS)
  g++ $(OBJECTS) -o GoRide

# Commands for partial compilation of c++ source files:
Bike.o: Bike.cpp Bike.h Wheel.h
  g++ -c Bike.cpp

Tricycle.o: Tricycle.cpp Tricycle.h Wheel.h
  g++ -c Tricyle.cpp

main.o: main.cpp Bike.h Tricycle.h
  g++ -c main.cpp
```

Note that the name that is given to a macro is chosen, like any other variable name, by the programmer. You may name it whatever you like. However, when selecting macro names you should apply the same rules as you would for variables in your programs. Most importantly they should be descriptive of what they represent. Cutomary to our coding standards, we treat these macros similar to named constants in C++ programs by using ALL_CAPS when defining them.

A.6.2 Special macros

In addition to those macros which you can create yourself, there are a few built-in macros which are used internally by the make program. Some of them are listed below:

CC	Contains the current compiler. Defaults to cc
CFLAGS	Special options which are added to the built-in compile rule
$@	Full name of the current target.
$?	A list of files for current dependency which are out-of-date.
$<	The source file of the current (single) dependency.

A.6.3 Rules

The real power of makefiles is seen when rules are used. A new % wild card character is introduced. The meaning of % is similar to *. However, it has different behaviour when expanding. Instead of putting all files that match the pattern in place in the single command, a separate command is created for each instance.

The following is an example of a rule that specifies that any .o file in the current folder depends on its corresponding .cpp file and can be created by compiling the .cpp file with the -c flag:

```
%.o:  %.cpp
      g++ $< -Wall -c -o $@
```

This compiling is specified to be done with warnings enabled. $< expands to the first dependency (a .cpp source file). $@ expands to the target (the corresponding .o file). This single rule can be used instead of a specific rule for each of the object files in the system.

Rules need not be for compiling only. For example, if we have a program called **dingamaging** that takes a **.dmg** file and automatically generates a **.cpp** file, we can automate the generation of .cpp files from .dmg files by adding the following rule to the makefile:

```
%.cpp:  %.dmg
      dingamaging $< -o $@
```

If we have added the above mentioned rule to our makefile and have a file named Bike.dmg in the current folder that is newer than the Bike.cpp file in the folder. The make program will generate a new Bike.cpp file, which in turn will trigger the recompilation of subsequent files depending on Bike.cpp.

In our final version of our example makefile below, we have expanded our use of macros. We redefine some of the pre-defined macros, for example CC and CFLAGS to contain detail specific to our needs. We also define a different flag set to be used when linking with a macro called LFLAGS. This way we exclude the flags that do not make sense when compiling from the CFLAGS list. We also include two custom commands that are independent of any other files (they have no dependancy lists). These are respectively to remove redundant files and to execute the program.

```
CC = g++
CFLAGS = -Wall -Werror
LFLAGS = -static
TARGET = GoRide
```

```
OBJECTS = Bike.o Tricycle.o main.o

# Linking all the object code:
all: $(OBJECTS)
  $(CC) $(LFLAGS) $(OBJECTS) -o $(TARGET)

# Dependencies:
Bike.o: Bike.h Wheel.h
Tricycle.o: Tricycle.h Wheel.h
main.o: Bike.h Tricycle.h

# Compilation rule:
%.o:  %.cpp
  $(CC) $< $(CFLAGS) -c -o $@

# Custom commands:
clean:
  rm -f $(TARGET) $(OBJS) *~ # deleting executable, .o's and backups

run:
  ./$(TARGET) # executing the program
```

The detail needed in the dependency list was reduced by the presence of the compilation rule. Because the rule specify that each .o file is dependant on its corresponding .cpp file, we no longer need to specify the .cpp files in the list of dependancies. We only need to list all the .h files that is included in the .cpp file to ensure proper partial recompilation if one of the .h files are changed. The rule also specifies how the .o files can be created. Therefore, this detail is not needed where the dependencies are listed. The make program will know *when* to create a specific .o file through the dependency list without the command, and will know *how* to create it through the specified compilation rule.

This generic makefile can now easily be modified to be applicibale to a specific system. One only have to update the macros defined at the beginning of the makefile and ofcourse the dependencies.

A.7 Common errors in makefiles

A.7.1 Syntax

- Failing to put a TAB at the beginning of commands. This causes the commands not to run.
- To put a TAB at the beginning of a blank line. This causes the make utility to complain that there is a "blank" command.
- Not hitting return just after the backslash, should you choose to use it. If the character directly before a newline is not the backslash the text on the next line is not interpreted as being a continuation of the previous line. It would be the same as not having the continuation character at all.

A.7.2 Dependencies

- Not including all dependencies.

  To create an .o file by compiling the corresponding .cpp file, the .o file is dependent on the .cpp file **as well as** all the header files that are either directly or indirectly included in the .cpp file.

A.7.3 Order of linking

- Listing the files in a link command in a wrong order.

  If Aaa.cpp directly or indirectly includes Bbb.h, then Bbb.o should appear before Aaa.o in a compile or link command that includes these .o files in its source file list.

A.8 Challenges

The following are practical assignments you can do to convince yourself that you understand the work. These are listed in order of increasing difficulty.

1. Write a custom rule to `tar` the .cpp, .h files and the Makefile in the current folder.
2. Write a custom command that will print all .cpp files that have changed since the last build.
3. Write a makefile that use a rule to generate the dependency list of the .cpp files in the current folder and include it automatically in the makefile[4]

[4]You will have to find out about including files in a makefile